Who Wrote It? Identifying LLMs from Their Responses

CMPSC 448 Midterm Project · Fall 2026

Team leader (individual project): Younsoo Park

Code, data-building scripts and all raw results: see the GitHub repository submitted with this report (`README.md` explains how to reproduce every number).

Abstract. We study whether the authoring LLM can be identified from its text. We built a balanced dataset of 14,466 `(LLM_name, input, output)` triples from the public AlpacaEval outputs of 18 models in six families (GPT, Claude, Llama, Qwen, Mistral, Google), all answering the same 805 prompts, and evaluated with prompt-grouped splits. A CNN and a BiLSTM trained from scratch identify the family of unseen-prompt responses with 54.2% and 53.0% accuracy (chance 16.7%); a TF-IDF baseline reaches 59.0%. Claude, Llama and Google are easy to recognise; GPT and Qwen are often confused. The prompt alone is useless on this data (chance accuracy) and adds little to the response, but in a simulated biased collection the prompt alone reaches up to 32.4%, exposing a common dataset confound. Fingerprints transfer to unseen prompt sources with a small loss and to coding prompts with a somewhat larger loss (RQ3). Formatting (markdown bold, lists, headers), length and stock phrases are the strongest interpretable signals. Retraining the CNN without formatting costs 10.9 points, while replacing every content word by a placeholder changes accuracy by only +0.5: the fingerprint lies in how answers are structured, not in what they are about (RQ4).

1. Problem Definition

Given a piece of text produced by a large language model, we want to predict *which* LLM family produced it. We cast this as 6-way single-label text classification over the families **GPT** (OpenAI), **Claude** (Anthropic), **Llama** (Meta), **Qwen** (Alibaba), **Mistral** (Mistral AI) and **Google** (Gemini and Gemma). Each example is a triple `(LLM_name, LLM_input, LLM_output)`. The classifier sees the output (RQ1), the input, or both (RQ2), and must output the family. We report accuracy and macro-F1; with balanced classes, chance accuracy is 1/6 = 16.7%.

Two design principles guided the whole project. (1) **No prompt leakage.** Because many models answer the same prompt, a random row-level split would put the same question in train and test, and a model could memorise topic words instead of style. All our splits are made *by prompt*, so every test prompt is unseen. (2) **Control the prompt.** If different LLMs were queried with different prompts, a classifier could succeed by recognising the prompt distribution rather than the model. We therefore chose a dataset in which every model answered *exactly the same prompts*, which makes RQ2 a clean test.

2. Dataset Curation

Source. We use the public model outputs released with the AlpacaEval benchmark [1] (`github.com/tatsu-lab/alpaca_eval`, files `results/<model>/model_outputs.json`, commit `cd543a1`). AlpacaEval asks every model to answer the same 805 instructions, which were pooled from five public instruction sets: *helpful_base* (129 prompts), *koala* (156), *oasst* (188), *selfinstruct* (252) and *vicuna* (80). Each JSON record already contains the instruction, the output and the generator name, so it maps directly to `(LLM_name, LLM_input, LLM_output)`; we additionally keep the instruction source, which we use for RQ3. The outputs were produced by the benchmark maintainers or model submitters through the official APIs or the released model weights. The prompts are public benchmark instructions, so the data contains no private or personal information.

Selection and balancing. To capture a family-level fingerprint rather than one checkpoint, we picked three official (non-community-fine-tuned) models per family, spanning different sizes and release dates (Table 1). Empty responses were removed (24 rows: 23 from Gemma/Gemini, 1 missing Llama-2 output), giving **14,466 examples**;

the six classes contain 2,392 to 2,415 examples each, so the data is balanced by construction. The script `src/build_dataset.py` rebuilds the CSV from the raw files.

Table 1. Models used (18 models, 6 families, 805 shared prompts each).

Family	Models	Responses	Mean words
GPT	gpt-3.5-turbo-0613, gpt-4o-2024-05-13, gpt4_1106_preview	2,415	261
Claude	claude-2.1, claude-3-5-sonnet-20240620, claude-3-opus-20240229	2,415	198
Llama	Meta-Llama-3-70B-Instruct, Meta-Llama-3.1-405B-Instruct-Turbo, llama-2-70b-chat-hf	2,414	291
Qwen	Qwen1.5-110B-Chat, Qwen1.5-72B-Chat, Qwen2-72B-Instruct	2,415	244
Mistral	Mistral-7B-Instruct-v0.2, Mixtral-8x7B-Instruct-v0.1, mistral-large-2402	2,415	233
Google	gemini-pro, gemma-2b-it, gemma-7b-it	2,392	182

Splits. We split the 805 prompts (not the rows) into 70% train / 10% validation / 20% test with a fixed seed: 563 / 81 / 161 prompts, i.e. 10,114 / 1,456 / 2,896 responses. Because every prompt is answered by all 18 models, each split is also balanced across families, and the test prompts were never seen during training or model selection. Figure 1 shows the class sizes and the length distribution. Table 2 shows how differently the families answer one test prompt.

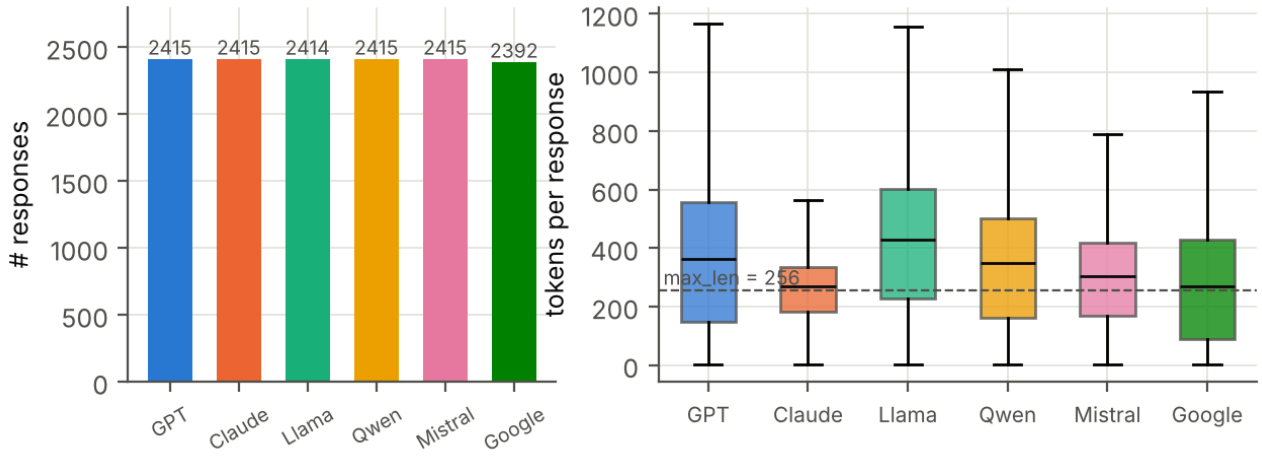


Figure 1. Left: responses per family. Right: response length in tokens (our tokenizer; whiskers at 1.5 IQR). The dashed line is the 256-token input length used by the neural models.

Table 2. Beginnings of four responses to the same prompt (a request for an Indonesian recipe). The opening phrase and the formatting already differ by family.

gpt4_1106_preview (GPT)	Tahu Gejrot Cirebon is a popular Indonesian street food originating from Cirebon, a port city in West Java, Indonesia. It's a sweet, spicy, and sour dish made w...
claude-3-opus (Claude)	Certainly! Here's a recipe for Tahu Gejrot Cirebon, a popular Indonesian dish from the city of Cirebon in West Java: Ingredients: - 500g fried tofu, cut into b...
Llama-3-70B-Instruct (Llama)	What a great choice! Tahu Gejrot Cirebon is a popular Indonesian street food originating from Cirebon, West Java. It's a delicious and refreshing dish that's pe...
gemini-pro (Google)	**Tahu Gejrot Cirebon** **Ingredients:**

* 20 pieces of fried tofu
* 100 grams of red chilies, seeded and chopped
...

---

3. Models and Training

Tokenisation and input representation. Formatting is likely part of an LLM's style, so we did not use a standard lower-casing word tokenizer. Our regex tokenizer (`src/common.py`) keeps letter case, splits every punctuation and markdown symbol into its own token (`*`, `#`, `-`, `:` ...) and turns every line break into a special `<nl>` token. The vocabulary is built on the training split only (tokens with frequency ≥ 2, capped at 30,000; about 28,723 types; unseen tokens map to `<unk>`). Each response is truncated to its first 256 tokens (the median response is about 310 tokens and 62% of responses are longer, Figure 1); we study the effect of this length in RQ4. For input + output (RQ2), the sequence is `[input tokens (≤64)] <sep> [output tokens]`, truncated to 256 tokens.

CNN. A Kim-style text CNN [2]: 128-d embeddings learned from scratch (embedding dropout 0.2) → three parallel 1-D convolutions with kernel widths 3, 4 and 5 and 128 filters each, ReLU → global max-pooling over time (padding positions are masked) → dropout 0.5 → linear layer to 6 logits. The kernels act as learned n-gram detectors (for example "`Certainly ! Here`" or "`<nl> * *`"), and max-pooling makes the detection position-independent. Total: 3.88M parameters (most of them in the embedding table).

RNN (BiLSTM). 128-d embeddings (dropout 0.2) → one bidirectional LSTM layer with 64 hidden units per direction, run on packed sequences so padding is skipped → concatenation of mean-pooling and max-pooling of the hidden states over time (256-d) → dropout 0.4 → linear layer. We pool over all time steps instead of using only the final state because the fingerprint (an opener, a closing offer) can sit anywhere in the response. Total: 3.78M parameters. We used 64 hidden units instead of 128 because a 128-unit LSTM took about 3.6 minutes per epoch on our 2-core CPU, which made the required number of runs infeasible; the 64-unit model takes about 1.8 minutes.

Training. Both models are trained with cross-entropy loss, AdamW (weight decay 1e-4; learning rate 1e-3 for the CNN and 2e-3 for the BiLSTM), batch size 64, and gradient-norm clipping at 1.0. We select the checkpoint with the best validation macro-F1 and stop early after 3 epochs without improvement (maximum 12 epochs for the CNN; 8 for the main BiLSTM run and 6 for the other BiLSTM runs). With seed 0 the CNN trained for 9 epochs (3.5 min) and the BiLSTM for 8 epochs (20.6 min) on CPU. Figure 3 shows the learning curves. Our first CNN configuration (learning rate 2e-3 and dropout 0.5 applied directly to the embeddings) was unstable and stopped at 43% accuracy; lowering the embedding dropout and the learning rate fixed this. All other experiments reuse these exact settings, and all experiments use the same prompt split.

Baselines. To put the neural numbers in context we also train (i) a TF-IDF (word 1-2 grams, case-sensitive, punctuation kept) + logistic regression classifier and (ii) a gradient-boosting classifier on 28 hand-crafted stylometric features (Section 7).

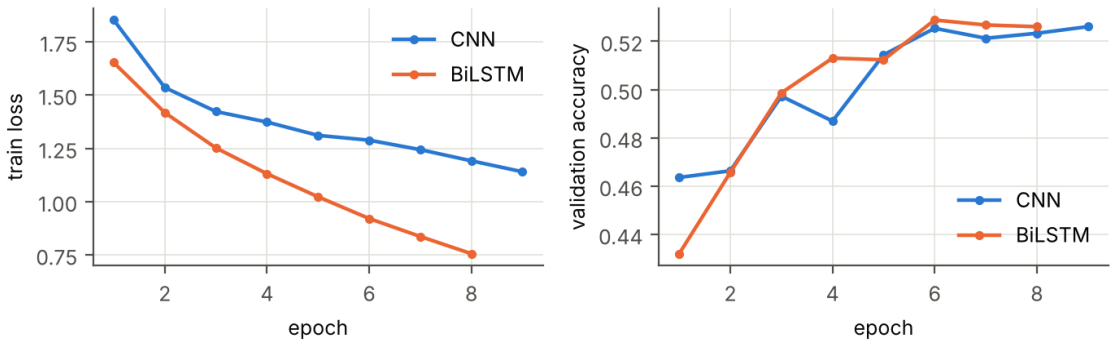


Figure 3. Training loss and validation accuracy per epoch (seed 0). The selected checkpoint is the epoch with the best validation macro-F1.

4. RQ1: Can we identify which LLM generated a response?

Table 3. RQ1: family identification from the response only (test set: 2,896 responses to 161 unseen prompts). Mean ± std over seeds.

Model	Accuracy (%)	Macro-F1 (%)	Seeds	Params
CNN (ours)	54.2 ± 1.4	53.0 ± 1.7	3	3.88M
BiLSTM (ours)	53.0	51.6	1	3.78M
TF-IDF (1-2 gram) + logistic regression	59.0	58.3	-	-
Stylometric features only (gradient boosting)	55.8	54.7	-	28 feats
Chance (6 balanced classes)	16.7	-	-	-

Yes, well above chance. From the response alone the CNN reaches 54.2% accuracy and the BiLSTM 53.0%, about 3.2 times chance (16.7%), on prompts never seen in training (Table 3). The two architectures are close; the CNN is slightly better and about 6 times faster to train on CPU. The CNN's standard deviation over 3 seeds (1.4 points) is larger than the CNN/BiLSTM gap, so we do not claim one architecture is clearly better (the BiLSTM was run with 1 seed because of its training cost). A plain TF-IDF + logistic-regression model is stronger (59.0%). The linear model sees the *whole* response while our networks see only the first 256 tokens, but Section 7.3 shows that the CNN is already saturated at 128 tokens, so truncation is not the main reason. More likely, with only ~10k training texts, embeddings and filters learned from scratch are data-hungry, whereas sparse n-gram counts with a convex classifier are a very strong and sample-efficient model of style.

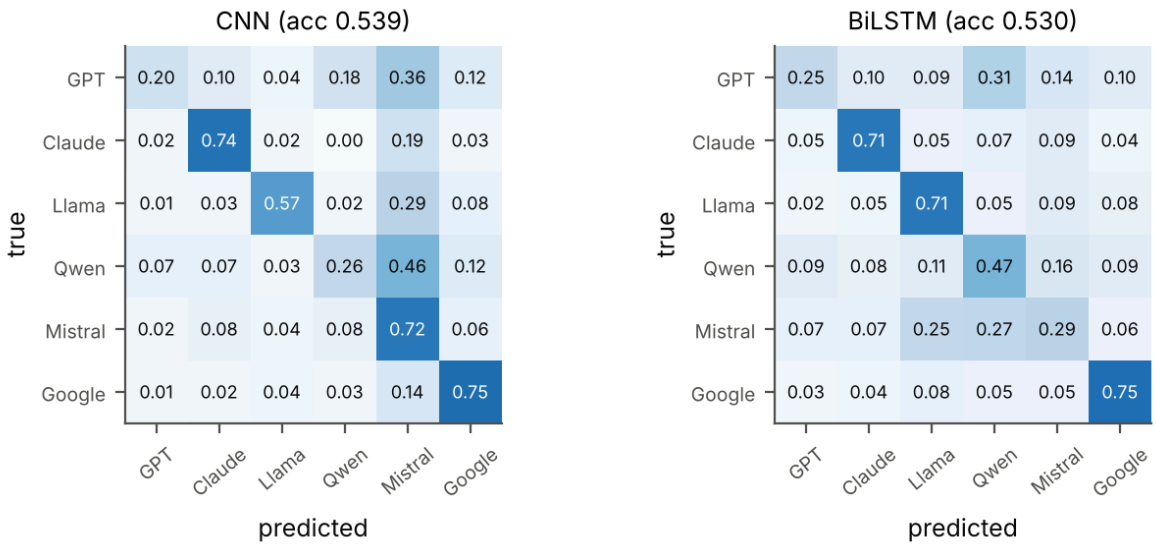


Figure 2. Row-normalised confusion matrices on the test set (seed 0). Diagonal = recall per family.

Which families are easy? Table 4 and Figure 2 show a clear split. **Claude, Llama and Google** are identified with F1 between 0.62 and 0.76 by all three classifiers, while **GPT and Qwen** are the hardest (F1 between 0.32 and 0.46) and are mostly confused with each other and with Mistral. For example, the BiLSTM labels 31% of GPT responses as Qwen. A plausible explanation (which we cannot verify from this data) is that GPT-style chat answers were widely used to train open models, so Qwen and Mistral answers look GPT-like. Section 7 also shows that Qwen and Mistral have no feature that stands out strongly, and GPT's most distinctive habits (markdown headers, "Certainly!") occur only in a minority of its answers, while Claude (almost no markdown bold, "Here is/are" openers), Llama ("What a great ...!", many exclamation marks) and Google (bulleted * lists, a bold title as the first line) each have a distinctive signature. The CNN also over-predicts Mistral (36% of its predictions vs. 16.7% true share), i.e. Mistral acts as its "default" class for generic-looking answers, which hurts GPT and Qwen recall.

Table 4. Per-family F1 (%) on the RQ1 test set (CNN/BiLSTM averaged over seeds).

Model	GPT	Claude	Llama	Qwen	Mistral	Google
CNN	32.2	71.1	66.0	35.6	43.3	69.5
BiLSTM	33.7	69.0	62.3	42.2	31.9	70.8
TF-IDF+LR	45.1	71.3	64.3	45.9	47.2	75.7

Model-level (18-way) identification. As an extra test we trained the same CNN to predict the exact model among all 18. It reaches 36.1% accuracy (chance 5.6%), and 30% of its mistakes confuse two models of the *same* family (a random error would hit the same family only 2/17 = 12% of the time). So sibling models really do share a style, which supports treating the family as the label.

5. RQ2: Does the user's prompt help identify the LLM?

5.1 Input only, output only, input + output

Table 5. RQ2: accuracy (%) with input only, output only, and input + output. All runs use seed 0. Left: our AlpacaEval dataset. Right: a simulated biased collection (Section 5.2). Chance = 16.7.

Model	AlpacaEval (shared prompts)			Biased collection		
	Input	Output	In+Out	Input	Output	In+Out
CNN	16.7	53.9	56.2	23.3	48.5	51.5
BiLSTM	16.7	53.0	53.1	27.0	44.3	43.0
TF-IDF+LR	16.7	59.0	58.9	32.4	54.0	55.8

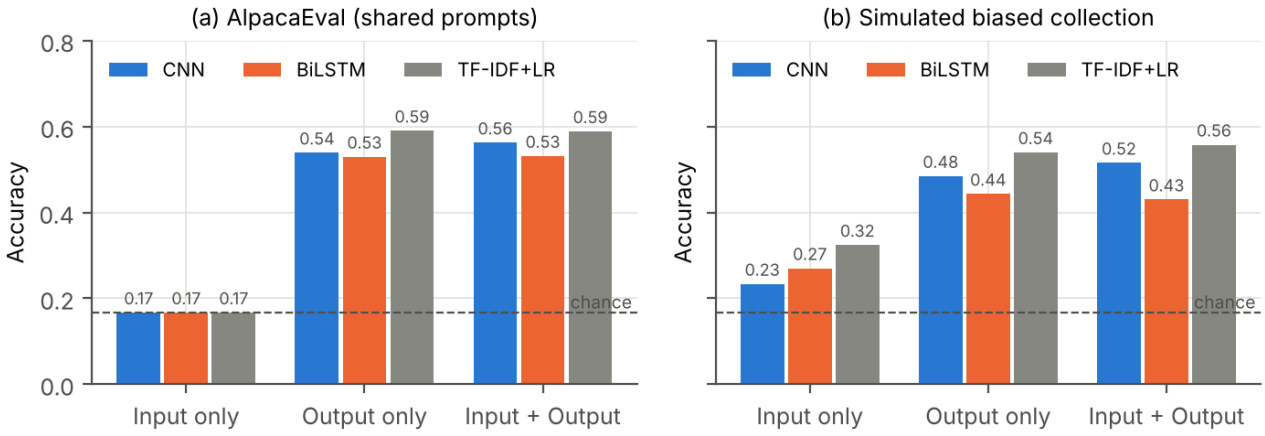


Figure 4. RQ2 accuracy. (a) On AlpacaEval every model answered the same prompts, so the input alone carries no information. (b) When each family is queried mostly with prompts of one topic (simulated biased collection), the input alone becomes predictive.

i) Can the LLM be predicted from the prompt alone? Not on our dataset: every input-only classifier sits at exactly chance (16.7% CNN, 16.7% BiLSTM, 16.7% TF-IDF). This is the expected outcome of our design: each of the 805 prompts appears once per model, i.e. three times with each family label, so the conditional distribution of the label given the prompt is uniform and no classifier can do better than guessing (the models collapse to predicting mostly two or three classes, hence the low macro-F1).

ii) Does adding the input help? Only a little. For the CNN, input + output gives 56.2% vs. 53.9% for output only; for the BiLSTM 53.1% vs. 53.0%; for TF-IDF the two are equal (58.9% vs. 59.0%). Since the prompt itself is uninformative here, any gain must come from *interactions*: the prompt tells the model what kind of answer to expect (a list request, a coding question, a short factual question), so it can judge whether the formatting or length of the answer is unusual for that request. The effect is small and comparable to seed-to-seed variation, so we treat it as suggestive rather than conclusive. Note also that adding up to 64 prompt tokens leaves fewer of the 256 tokens for the response.

5.2 iii) When would input-only classification work "surprisingly well"?

Many public "which-LLM" datasets are collected differently from ours: each LLM is queried by different people, at different times, or with different task lists. Then the prompt distribution itself differs by label, and an input-only classifier can succeed without ever seeing an LLM's text. To demonstrate this, we built a **simulated biased collection**: we clustered the 805 prompts into 6 equal-size topic clusters (TF-IDF + k-means with balanced assignment) and paired each cluster with one family; a (prompt, family) response is kept with probability 0.9 if they are paired and 0.15 otherwise. This gives 4,091 responses in which each family is still answered by real LLM text, but 54% of a family's examples come from "its" topic. With exactly the same pipeline and prompt-grouped split (test: 844 responses), the input-only models now beat chance clearly: TF-IDF 32.4%, CNN 23.3% and BiLSTM 27.0%. These classifiers have learned "who was asked about what", not how the LLMs write. Output-only accuracy is lower than on the full dataset because there is about 3.5 times less training data. Adding the input

helps the CNN (51.5% vs. 48.5%) and TF-IDF (55.8% vs. 54.0%) here, but not the BiLSTM (43.0% vs. 44.3%), although its validation accuracy was higher with the input (48.0% vs. 45.1%). With only 844 test responses, differences of 1 to 2 points are within noise (one standard error is about 1.7 points). **Lesson:** when input-only works well, it signals a confound in data collection (topic, time period, user population or language), and output-based results on such data are inflated by the same confound because the response also mentions the topic. A shared-prompt design like ours removes it.

6. RQ3: Do LLM fingerprints generalise across tasks or domains?

Setup. We use two kinds of domain shift. (a) **Leave-one-source-out (LOSO):** the 805 prompts come from five instruction sets with different styles (e.g. *selfinstruct* holds short, task-like instructions such as "Rewrite this sentence ..."; *vicuna* and *koala* hold open-ended chat questions; *oasst* and *helpful_base* come from real user conversations). For each source we train on the other four (with a 12% prompt-grouped validation split) and test on all responses from the held-out source. (b) **General → coding:** we mark the 79 prompts that ask for code (regex on words such as *code, python, function, SQL, script, algorithm*), train on all non-coding prompts and test on the coding prompts. This mirrors the "train on general QA, test on coding" example in the project description. The in-domain reference is the RQ1 model evaluated on the test prompts of the same source (or on the coding prompts of the RQ1 test split).

Table 6. RQ3: accuracy (%) per prompt source. Each cell is *in-domain* (test prompts of that source, model trained on all sources) → *cross-domain* (model trained on the other four sources only). Last column: coding prompts; in-domain = the 17 coding prompts (306 responses) in the RQ1 test split (CNN: mean of seeds 1-2), cross-domain = trained on non-coding prompts only and tested on all coding prompts.

Model	helpful_base	koala	oasst	selfinstruct	vicuna	coding (held out)
CNN	63.3 → 58.0	50.8 → 53.0	53.1 → 52.0	52.1 → 47.6	52.2 → 54.7	51.3 → 45.3
TF-IDF+LR	69.5 → 64.5	59.1 → 58.0	59.7 → 58.1	52.4 → 46.7	63.9 → 61.7	51.6 → 52.8

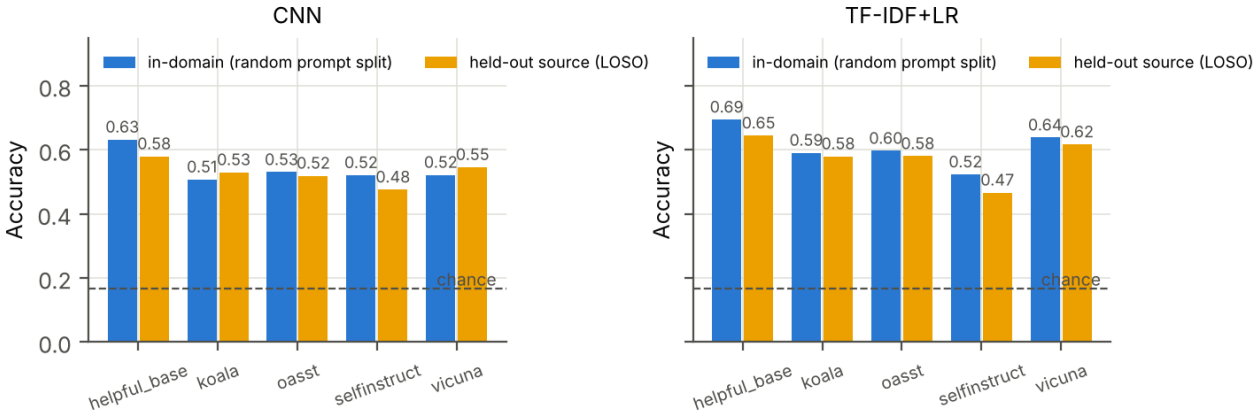


Figure 5. RQ3 in-domain vs. held-out-source accuracy per source (CNN and TF-IDF baseline).

i) Does performance decrease on unseen tasks? Only slightly. Averaged over the five held-out sources, the CNN goes from 54.3% in-domain to 53.1% cross-domain, and TF-IDF from 60.9% to 57.8% (Table 6, Figure 5). The largest drops are on *helpful_base* (CNN -5.2, TF-IDF -5.0 points) and *selfinstruct* (CNN -4.5, TF-IDF -5.7). *helpful_base* is the easiest source in-domain, so part of its advantage was source-specific. *selfinstruct* is the most distinctive source (short, templated tasks such as rewriting, classifying or extracting): its answers are short (median 140 words vs. 239 to 335 words for the other sources), so they carry fewer stylistic cues, and the other four sources contain fewer tasks of this kind. Two sources even improve slightly for the CNN (koala and vicuna), which shows

that most of the signal the model uses is not tied to one prompt source. Note that the cross-domain models are trained on a similar number of prompts as the in-domain model (487 to 638 training prompts vs. 563), so the comparison is largely not confounded by training-set size. For **coding** prompts, the CNN trained without any coding prompt reaches 45.3% (in-domain: 51.3%) and TF-IDF 52.8% (in-domain: 51.6%). Coding answers are harder for every model, even in-domain, likely because a large part of the answer is code, which looks similar across LLMs. Removing all coding prompts from training changes accuracy on them by -6.0 points for the CNN and +1.2 for TF-IDF (on a small test set of 306 responses). Even without any coding example the models stay far above chance, which suggests that much of the fingerprint lives in the natural-language framing around the code (openers, explanations, closing remarks).

ii) Are some models more consistently identifiable? Yes. Averaged over the five LOSO folds, the per-family F1 of the CNN is: Google 0.71, Claude 0.65, Llama 0.63, Mistral 0.43, GPT 0.39, Qwen 0.33. The grouping is the same as in-domain (Table 4): the families with strong formatting habits transfer well, while GPT and Qwen remain hard everywhere.

iii) General fingerprint or task-specific patterns? Mostly a general fingerprint. If the classifier relied on task-specific patterns, accuracy would collapse on an unseen source; instead it stays at roughly 3 times chance on every held-out source and on coding. The remaining gap shows that part of the signal is task-dependent: how a model formats an answer depends on the task (a list for "give me ideas", a code block for programming), so a model trained without a task type has not seen how each LLM handles it.

7. RQ4: What characteristics distinguish different LLMs?

7.1 Descriptive statistics

We computed 28 interpretable features per response (length, paragraphs, list and header usage, markdown bold, code blocks, sentence and word length, type-token ratio of the first 100 words, punctuation rates, and opening/closing phrases). Figure 7 shows the family means. The families differ in clear and readable ways:

- **Length:** Llama (291 words on average) and GPT (261) write the longest answers, Claude (198) and Google (182) the shortest.
- **Markdown bold:** Claude essentially never uses `**bold**` (0.005 per response) while Google, Llama, GPT and Qwen use about 3 to 5 per response. Mistral uses little (0.54).
- **Lists and headers:** Google prefers bullet lists (31% of lines) over numbered lists; GPT is the only family that regularly uses markdown headers (`###`; 0.67 per response).
- **Openers:** 32% of Claude answers start with "Here ...", 12% of GPT answers with "Certainly/Absolutely/Of course", 21% of Google answers with a bold title or header, and Llama has the most exclamation marks ("What a great question!") (Table 7).
- **Lexical diversity** differs only slightly (type-token ratio of the first 100 words: 0.66 to 0.70).

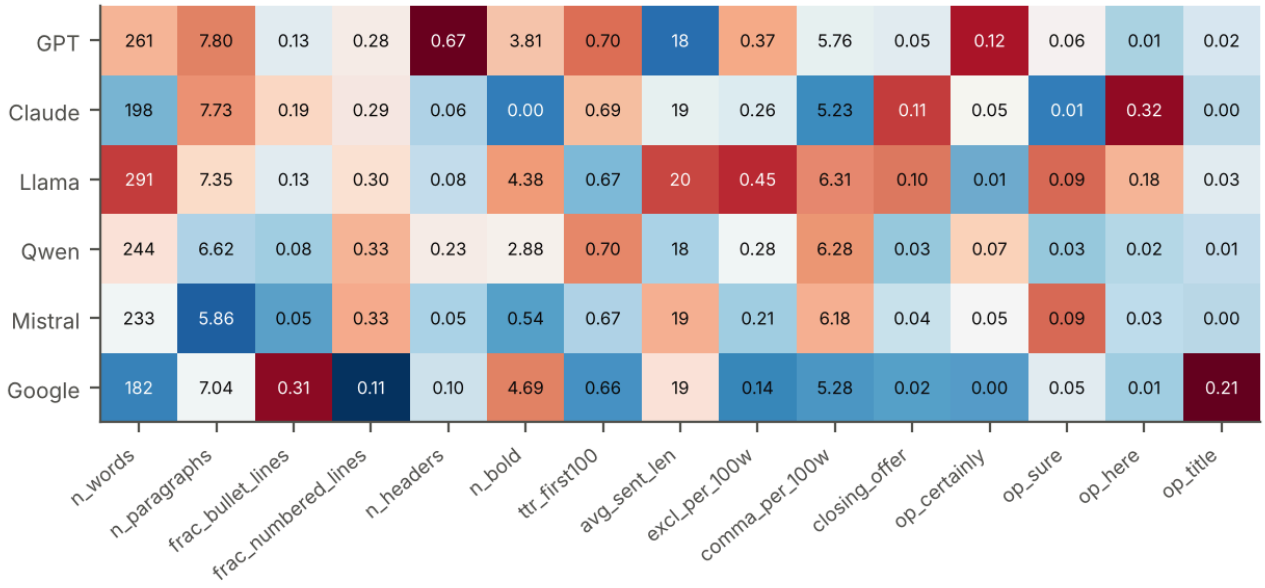


Figure 7. Mean value of selected stylometric features per family (cell colour = z-score across families; red = above average, blue = below).

Table 7. RQ4: most frequent opening tokens and the n-grams with the largest positive logistic-regression weight for each family (output-only TF-IDF model).

Family	Top openings (share)	Most indicative n-grams
GPT	`Certainly !` (8%), `Sure !` (3%), `Sure ,` (2%)	`# #`, `- *`, `’`, `Sure !`, `Certainly !`, `* -`, `Certainly`, `—`, `! Here`, `’ s`
Claude	`Here are` (12%), `Here is` (11%), `Here '` (9%)	`: -`, `Here`, `: 1`, `:`, `. -`, `Here is`, `Here '`, `key`, `I apologize`, `-`
Llama	`Here is` (7%), `Sure ,` (7%), `Here are` (6%)	`!`, `*`, `* 1`, `Here`, `) *`, `, and`, `: *`, `, such`, `, ,`, `Here is`
Qwen	`1 .` (7%), `Certainly !` (4%), `Sure ,` (3%)	`* :`, `but`, `Remember ,`, `might`, `Remember`, `and the`, `, and`, `. Remember`, `* -`, `- *`
Mistral	`Sure ,` (9%), `1 .` (6%), `Title :` (4%)	`a simple`, `help`, `Absolutely`, `to help`, `This`, `Sure ,`, `. 1`, `Title :`, `d be`, `be happy`
Google	`* *` (21%), `1 .` (10%), `Sure ,` (5%)	`: *`, `*`, `* *`, `. *`, `* The`, `. -`, `The answer`, `It is`, `answer`, `$ $`

7.2 Which signals does a classifier use?

A gradient-boosting classifier trained *only* on the 28 hand-crafted features reaches 55.8% accuracy, already above our neural models and close to the TF-IDF model. Using one feature group at a time: formatting features alone give 41.9%, length features alone 35.8%, and lexical/phrasal features alone 36.8%. The most important features by permutation importance (accuracy drop in points when the feature is shuffled) are `n_bold` (11.9), `n_paragraphs` (8.0), `frac_numbered_lines` (6.4), `n_words` (5.4), `op_here` (4.2), `n_chars` (3.6). So the fingerprint is **both structural and linguistic**, but structure (formatting and length) is the single strongest group. The TF-IDF n-grams in Table 7 tell the same story: the top weights are markdown patterns (`# #`, `* *`, `: -`) and stock phrases ("Certainly !", "Here is", "I hope", "Remember ,", "happy to help", "The answer is").

7.3 Does removing signals reduce performance? (ablations)

Table 8. RQ4 ablations: the text is transformed in train, validation and test, and the model is retrained from scratch (accuracy %, Δ = change vs. original). BiLSTM ablations were not run (compute budget).

Input text	CNN acc	Δ
Original text	53.9	
Formatting removed (markdown symbols, list markers, line breaks)	43.0	-10.9
First line / first sentence removed	48.8	-5.1
Function words + punctuation only (content words → `W`)	54.4	+0.5
Content words only (lower-cased, no stop words, no punctuation, no line breaks)	27.1	-26.8
Word order shuffled within each response	40.7	-13.2

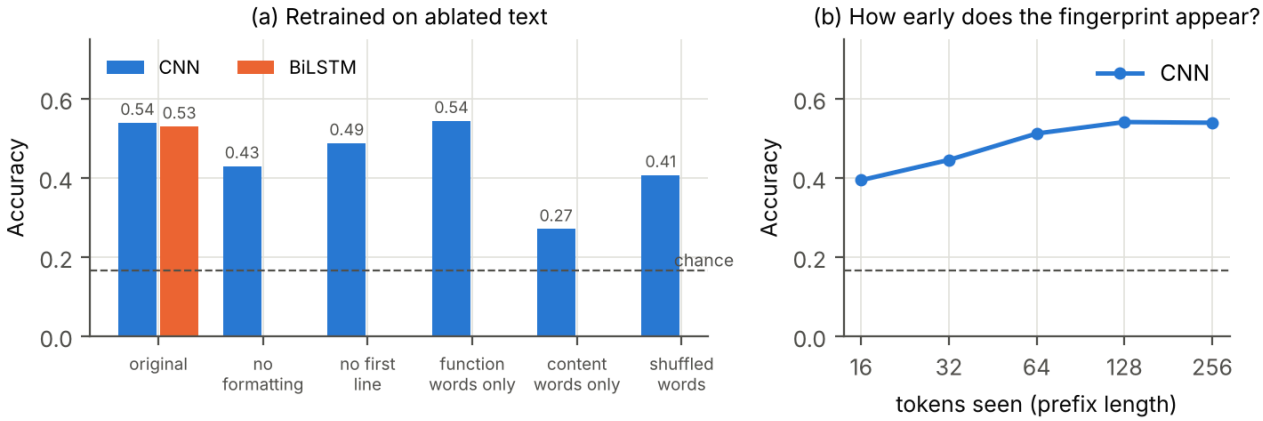


Figure 6. (a) Accuracy after retraining on transformed text. (b) Accuracy when the model only sees the first k tokens of each response (CNN retrained for each k).

To test causally which signals matter, we transformed the text and **retrained** the CNN from scratch on the transformed data (Table 8, Figure 6a). Retraining matters: simply feeding altered text to the original model would mainly measure distribution shift, whereas retraining asks whether the information is still *learnable*.

- **Removing formatting** (markdown symbols, list markers and all line breaks) costs 10.9 points, a large drop. Formatting is a major but not the only signal: the model still reaches 43.0% (about 2.6 times chance) on plain running text.
- **Keeping only function words and punctuation** (every content word replaced by the same placeholder `W`) changes accuracy by only +0.5 points, well within the CNN's seed-to-seed variation. What a response is *about* is almost irrelevant; how it is *put together* (function words, punctuation, line structure, sentence rhythm) carries nearly all the information the CNN uses.
- **Keeping only content words** (lower-cased, no stop words, no punctuation, no line breaks) gives 27.1% (-26.8), the largest drop of all. This model learned slowly (training loss was still 1.71 when early stopping triggered), so the number is a lower bound, but vocabulary choice alone is clearly a much weaker signal than structure.
- **Removing the first line or sentence** changes accuracy by -5.1 points. Stock openers ("Certainly!", "Here are", "What a great question!") are very visible in Table 7, but the model can recover most of the signal from the rest of the response.
- **Shuffling the word order** costs 13.2 points: local order (the n-grams detected by the convolution filters) matters, but a bag of tokens still identifies the family 2.4 times better than chance.

How early does the fingerprint appear? Retraining the CNN on only the first k tokens of each response gives: 16 tokens: 39.4%, 32 tokens: 44.5%, 64 tokens: 51.2%, 128 tokens: 54.1%, 256 tokens: 53.9% (Figure 6b). Even the first 16 tokens (roughly the opening sentence) identify the family at more than twice chance, most of the gain is reached by 64 tokens, and the curve is flat between 128 and 256 tokens. The fingerprint is therefore front-loaded (openers, the first heading or list), and longer CNN inputs are unlikely to help much.

Summary for RQ4. i) The classifier relies on formatting habits (bold, lists, headers, line breaks), response length and characteristic phrases, and on the arrangement of function words and punctuation rather than on topic words. ii) The signals are both structural and linguistic: structure is the strongest single group, but removing all formatting still leaves a clear linguistic fingerprint. iii) Removing formatting reduces accuracy substantially (by 10.9 points), while removing content words barely matters.

8. Lessons and Experience

- **The data design decides what a result means.** Our most important decision was not an architecture but the dataset: using shared prompts and prompt-grouped splits. Section 5.2 shows that with a biased collection, a classifier can look clever while only recognising the topic. Before trusting any "which-LLM" accuracy we would now always ask: can the input alone predict the label? If yes, the dataset has a confound.
- **Prompt leakage is easy to miss.** Every prompt appears 18 times in our data. A random row split would have put the same question (and therefore the same topic words) in train and test. Grouping by prompt is a one-line change (`GroupShuffleSplit`) but it is essential for an honest test set.
- **Strong simple baselines are humbling and useful.** TF-IDF + logistic regression beat both of our neural networks, and 28 hand-crafted features were competitive with them. Without the baselines we would have reported ~54% as a good number without context. With ~10k training texts and embeddings learned from scratch, a CNN or LSTM has no advantage over sparse n-grams; we expect pretrained encoders (e.g. a fine-tuned RoBERTa) to close the gap.
- **Debugging training matters more than architecture tweaks.** Our first CNN used dropout 0.5 directly on the embeddings and a learning rate of 2e-3; it stopped after 3 epochs at 43% accuracy. Lowering the embedding dropout to 0.2 and the learning rate to 1e-3 gave +11 points. We learned to look at the learning curves (its validation F1 jumped between 0.31 and 0.39 from epoch to epoch, a clear warning sign) before changing the model.
- **Compute is a design constraint.** All training ran on a 2-core CPU. The BiLSTM was about 4 to 6 times slower per epoch than the CNN, so we halved its hidden size, capped its epochs (8 for RQ1, 6 elsewhere) and ran the RQ3/RQ4 sweeps with the CNN only; we ran 3 seeds only for the main CNN result (RQ1). We built the experiment runner so each experiment writes its own JSON file and the queue can resume after an interruption, which saved us when our machine restarted in the middle of a run.
- **Interpretability needs explicit experiments.** Accuracy alone did not tell us *why* the models work. The stylometric analysis, the n-gram weights and especially the retraining ablations (removing formatting, keeping only function words, shuffling words) were what turned the classifier into an explanation.
- **Limitations.** AlpacaEval outputs come from 2023 to 2024 model versions answering general instructions in English with default settings, so our fingerprints may not transfer to newer versions, other languages, system prompts that request a style, or edited text. Our coding subset is small (79 prompts), and most experiments use one seed.

References

[1] X. Li, T. Zhang, Y. Dubois, R. Taori, I. Gulrajani, C. Guestrin, P. Liang, T. B. Hashimoto. AlpacaEval: An Automatic Evaluator of Instruction-following Models. GitHub repository, 2023. https://github.com/tatsu-lab/alpaca_eval

[2] Y. Kim. Convolutional Neural Networks for Sentence Classification. EMNLP 2014.
[3] S. Hochreiter and J. Schmidhuber. Long Short-Term Memory. Neural Computation, 9(8), 1997.
[4] F. Pedregosa et al. Scikit-learn: Machine Learning in Python. JMLR 12, 2011.
[5] A. Paszke et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. NeurIPS 2019.

Use of AI tools. As permitted by the project description, an LLM assistant was used to help write and debug code and to draft text. All experiments were run, checked and interpreted by the author, who is responsible for the content.